MAGIS-AUTO
Plan de trabajo para los tres integrantes

Asignación detallada, entregables, pruebas, comentarios de código y publicación en Git. Preparado el 13 de octubre de 2026 a partir del trabajo y las evidencias compartidas en el chat.

El acuerdo principal

Cada integrante tiene una responsabilidad principal, pero **los tres deben entender el recorrido completo**: cliente, crédito, aprobación, cuotas, pago, anulación y reportes. Cada uno debe explicar con sus propias palabras lo que revisa y comenta.

Responsable	Trabajo principal	Entrega verificable
Compañero 1	Base de datos, SQL y reglas del negocio.	Diagrama, inventario de procedimientos, SQL comentado, cálculos y hallazgos.
Compañero 2	Pruebas, evidencias y guía de uso.	Matriz de pruebas, capturas organizadas, errores reproducibles y manual.
Diego	API, interfaz e integración.	Código revisado y comentado, correcciones integradas y versión comprobada.

Punto de partida

El proyecto ya está avanzado. La tarea es revisar, entender, documentar y corregir lo necesario. En el chat se completó el recorrido manual de las 20 consultas y varios casos límite. Esto no equivale a cerrar toda la fase 6 ni a certificar la seguridad del sistema.

Antes de que los compañeros creen sus ramas, Diego debe confirmar que la versión acordada, incluidos los reportes recientes, está publicada en **pruebas-diego**. Compartirá el identificador del commit de partida. Este PDF no publica cambios ni asigna permisos en GitHub.

Orden de trabajo

1. Preparar una versión común y el entorno de validación.
2. Revisar el funcionamiento antes de editar.
3. Documentar hallazgos y coordinar las correcciones.
4. Comentar SQL y código después de comprenderlos.
5. Probar, subir las ramas y solicitar revisión.
6. Integrar y practicar juntos la exposición.

Regla del equipo: cada compañero usa su propia rama; Diego integra en pruebas-diego. Ninguno modifica main como parte de este trabajo.

Compañero 1: estructura y procedimientos

Objetivo: comprobar que la base de datos aplica las reglas del negocio y dejar una explicación que el equipo pueda defender.

1. Revisar tablas y relaciones

• Identificar clientes, fiadores, vendedores, agencias y vehículos; relacionarlos con créditos, cuotas, pagos y componentes del pago.

• Explicar claves primarias, claves foráneas, restricciones y datos obligatorios. Identificar qué evita duplicados o relaciones inválidas.

• Explicar qué registros conservan el historial y qué operaciones no deben resolverse eliminando datos.

Entrega: diagrama actualizado, contrastado con el SQL existente, y descripción de las relaciones principales. No dibujar relaciones que el esquema no tenga.

2. Seguir el recorrido de un crédito

Operación	Qué debe localizar y explicar
Crear o modificar borrador	Datos recibidos, validaciones, tablas afectadas y restricciones de edición.
Presentar y aprobar	Requisitos de presentación, comprobaciones de aprobación y generación de cuotas.
Registrar abono parcial	Saldo permitido, reparto entre componentes y emisión del recibo.
Completar una cuota	Cómo se identifica el pago completo y cómo se actualiza el saldo.
Anular un pago	Conservación del historial, reversión de efectos y recuperación del saldo.

Entrega: inventario con nombre real del procedimiento, archivo, parámetros, finalidad, validaciones, tablas que modifica y resultado devuelto. Usar referencias a archivos y funciones existentes; no inventar nombres de procedimientos.

3. Revisar las consultas

Relacionar las 20 consultas con sus procedimientos y resultados. Explicar las fechas de corte, anulaciones, agrupaciones y valores nulos. Coordinar con el compañero 2 las discrepancias entre datos y pantalla.

Criterio de terminado: puede mostrar dónde se implementa cada operación y explicar una ejecución con datos de prueba, incluyendo una operación rechazada por validación.

Compañero 1: comentarios y revisión

Primero comprender; después comentar. Los comentarios deben explicar las decisiones y mantenerse coherentes con el comportamiento real.

4. Comentar SQL con información útil

En cada procedimiento relevante, indicar finalidad, parámetros, resultados, reglas y tablas afectadas. En bloques clave explicar cálculos, redondeo, transacciones, tratamiento de errores y conservación del historial.

```
-- Finalidad: [operacion que realiza]
-- Parametros: [nombre, significado y restricciones]
-- Devuelve: [filas, valores o resultado esperado]
-- Valida: [reglas del negocio]
-- Transaccion: [que cambios deben ocurrir juntos]
-- Errores: [motivos por los que rechaza la operacion]
```

Evitar comentarios que solo repitan la instrucción, como "hace un SELECT". Es más útil explicar por qué se excluye un pago anulado o por qué una valoración no reduce la deuda. No comentar cada línea trivial.

5. Revisar protección junto con Diego

• Compañero 1: comprobar los permisos efectivos del usuario de la API y revisar los procedimientos, incluido cualquier SQL dinámico que pudiera existir.

• Diego: revisar cómo la API recibe y valida los datos y los envía como parámetros. La interfaz no sustituye la validación del servidor.

• Comprobar que las consultas y los mensajes de error no expongan contraseñas ni información de conexión. Registrar evidencia de las comprobaciones realizadas.

Usar procedimientos almacenados no basta, por sí solo, para afirmar que no existe inyección SQL. La conclusión debe basarse en la implementación revisada y en pruebas acordadas dentro del entorno de validación.

6. Proponer cambios sin afectar los datos de demostración

Registrar problema, archivo o procedimiento, pasos de reproducción, impacto y propuesta. Separar comentarios de cambios funcionales. Probar correcciones en la base de validación y coordinar con Diego su aplicación.

No ejecutar el instalador completo sobre una base existente como si fuera una actualización. Si se requiere modificar el esquema, acordar un script de migración y su comprobación. Si existen archivos SQL generados, identificar primero su fuente para no perder cambios al regenerarlos.

Entrega final: SQL comentado, inventario, diagrama, cálculos comprobados y lista de hallazgos con evidencia; no una afirmación genérica de que "todo es seguro".

Caso de prueba que todos deben explicar

Referencia: PRUEBA-CREDITO-01, cliente 6, vendedor 16, fiador 4 y vehículo Nissan Versa. Importes según las capturas compartidas; deben volver a comprobarse si se modifican los datos.

Concepto	Importe / dato
Precio al contado / enganche	Q100,000.00 / Q20,000.00
Capital financiado / plazo	Q80,000.00 / 12 meses
Intereses del plan / total contractual de cuotas	Q5,519.56 / Q87,739.56
Primera cuota / vencimiento	Q7,311.63 / 12-11-2026
Pagos efectivos al 13-10-2026	Q1,500.00
Capital recuperado / pendiente	Q1,291.09 / Q78,708.91
Saldo contractual pendiente	Q86,239.56
Saldo pendiente de la primera cuota	Q5,811.63
Intereses cobrados / pendientes	Q170.96 / Q5,348.60
Comisión efectiva	Q30.00
Valor depreciado / margen frente al capital	Q85,000.00 / Q6,291.09

Cómo se llega a los Q1,500.00 efectivos

Recibo 1: Q1,000.00 en efectivo, vigente. Recibo 2: Q6,311.63, posteriormente anulado. Recibo 3: Q500.00 por transferencia ACH, vigente. Por eso Q1,000.00 + Q500.00 = Q1,500.00; el recibo anulado conserva su historial, pero ya no reduce el saldo actual.

Reparto de los pagos efectivos

Capital Q1,291.09 + interés Q170.96 + manejo Q5.13 + seguro de auto Q24.62 + seguro de vida Q8.20 = **Q1,500.00**. El abono no se aplica íntegramente a capital.

Diferencias que deben dominar

Capital pendiente: Q80,000.00 - Q1,291.09 = Q78,708.91. Saldo contractual: Q87,739.56 - Q1,500.00 = Q86,239.56. Primera cuota pendiente: Q7,311.63 - Q1,500.00 = Q5,811.63.

Comisión generada Q156.23 - revertida Q126.23 = Q30.00. El desembolso contractual total es Q20,000.00 + Q87,739.56 = Q107,739.56; no representa lo pagado hasta hoy. La depreciación estimada de Q15,000.00 no disminuye la deuda.

Compañero 2: pruebas y evidencias

Objetivo: comprobar que el sistema puede utilizarse correctamente y dejar resultados reproducibles, sin confundir evidencia previa con pruebas propias.

1. Preparar una matriz de pruebas

Campo	Contenido obligatorio
Identificador y módulo	Un código estable, por ejemplo PAG-01, y el módulo revisado.
Caso y preparación	Operación que se probará; datos, estado previo y entorno necesario.
Pasos	Acciones exactas que otra persona pueda repetir.
Esperado y obtenido	Qué debía pasar y qué ocurrió realmente.
Evidencia	Archivo o captura correspondiente; fecha y responsable.
Estado	Correcto, fallido o pendiente. Indicar el commit usado cuando sea posible.

Cubrir catálogos, personas, vehículos, créditos, pagos y reportes. Distinguir "ejecutado por mí" de "revisado mediante evidencia de Diego". Una captura de impresión no demuestra que se guardó un PDF o que se imprimió en papel.

2. Comprobar formularios y mensajes

• Campos obligatorios vacíos, duplicados y valores fuera de límites, donde apliquen.

• Búsquedas con resultados y sin resultados; edición y persistencia al volver a consultar.

• Cancelación sin guardar; rechazo de operaciones no permitidas con mensajes comprensibles.

• Tablas, botones, desplazamiento horizontal, textos y vistas previas de impresión.

Usar datos de prueba identificables y coordinar operaciones que cambien información. No alterar el crédito de demostración sin acuerdo ni limpiar registros ajenos. Para pruebas nuevas, usar el entorno de validación acordado.

3. Registrar errores que se puedan reproducir

Entrega por error: módulo, versión, datos utilizados, pasos, resultado esperado, resultado real, captura e impacto. Distinguir si impide trabajar, produce datos incorrectos o es un detalle de presentación.

Criterio de terminado: otra persona puede repetir los casos y encontrar las evidencias sin necesitar una explicación adicional del autor.

Compañero 2: cobertura y manual

4. Organizar pagos y casos límite

• Abono parcial, pago completo, edición de observación y anulación con recuperación de saldo.

• Rechazo de monto superior al saldo; referencia requerida en transferencia; búsqueda y recuperación del historial completo.

• Recibos, componentes, comisiones y vista previa de impresión.

• Fechas inválidas, períodos vacíos, corte anterior a aprobación y diferencia entre cero y "Sin dato".

• Mora el día del vencimiento y al día siguiente; coincidencia entre pagos, estado de cuenta y reportes.

Mantener el contexto: el 12-11-2026 no aparece mora; al 13-11-2026, con los movimientos registrados en la prueba, hay un día de atraso y Q5,811.63 vencidos. Las consultas a fechas futuras no predicen pagos que todavía no se registraron.

5. Crear un índice de evidencias

Propuesta: una carpeta por módulo, archivos con identificador de caso y un índice que enlace cada captura. Ejemplo: REP-02_mora_dia_siguiente.png. Conservar las capturas originales; si se hace un recorte para explicar algo, mantener también la original.

Las capturas deben permitir ver los filtros y el resultado. Si la tabla es ancha, incluir la parte izquierda y la derecha. No marcar como comprobada una columna que quedó fuera de la captura.

6. Preparar una guía de uso

Explicar cómo registrar un cliente, preparar y presentar un crédito, aprobarlo, recibir un pago parcial o completo, anularlo, consultar reportes e imprimir un comprobante o carta. Incluir requisitos previos, pasos, resultado y errores habituales.

7. Separar mejoras visuales de fallos funcionales

Detalles ya observados para revisar: algunos montos se parten en dos líneas en el historial de pagos y la carta de cumpleaños tiene bastante espacio superior en la impresión. Registrarlos y confirmar si se corrigen; no presentarlos como arreglados por este documento.

Entrega final: matriz de pruebas, carpeta e índice de evidencias, lista de errores y guía de uso. El compañero 2 organiza la documentación común; el compañero 1 aporta el contenido técnico de SQL y Diego el de API e interfaz.

Lista de las 20 consultas

Lista de control para la matriz. El recorrido visual se realizó en el chat; cada caso debe enlazarse con su evidencia y sus límites, sin convertirlo en una prueba exhaustiva de todos los escenarios.

N.º / consulta	Qué debe comprobarse
01. Créditos emitidos	Detalle, resumen, fechas inválidas y período vacío.
02. Cartera en mora	Sin mora al vencer; 1 día y Q5,811.63 al día siguiente.
03. Pagos por medio	Efectivo neto Q1,000.00 y ACH Q500.00.
04. Ventas por vendedor	Vendedor 16: 1 crédito y Q80,000.00 colocados.
05. Cumpleañeros y cartas	Mes 8, año 2026, edad 28, carta y vista previa; sin envío de correo.
06. Estado de cuenta	Resumen, 12 cuotas y 3 movimientos; efectividad al corte.
07. Plan de pagos	Condiciones, cuotas y saldos; antes de aprobación, saldos sin dato.
08. Comisiones	Q156.23 generados - Q126.23 revertidos = Q30.00.
09. Capital pendiente	Q80,000.00 - Q1,291.09 = Q78,708.91.
10. Intereses del contrato	Q5,519.56 - Q170.96 = Q5,348.60.
11. Próximos vencimientos	Corte 13-10-2026, 30 días; cuota 1, saldo Q5,811.63.
12. Abonos parciales	Cuota 1: Q1,500.00 pagados y Q5,811.63 pendientes.
13. Marca y línea	Nissan / Versa: 1 crédito, Q80,000.00 financiados.
14. Agencia y región	Agencia 1, Oriente, saldo Q86,239.56; vencido Q0.00 al 13-10.
15. Mora por rangos	Al 13-11: rango 01-30, 1 crédito, Q5,811.63 vencidos.
16. Garantía frente al capital	Valor Q85,000.00; margen Q6,291.09.
17. Depreciación y costo	Total con enganche Q107,739.56; pérdida estimada Q15,000.00.
18. Créditos por fiador	Fiador 4, 1 crédito; saldos coincidentes.
19. Recaudación por componentes	Los cinco componentes netos suman Q1,500.00.
20. Pagos anulados	Importe Q6,311.63, comisión Q126.23, motivo y responsable.

Usar el corte 13-10-2026 salvo las excepciones indicadas. Para reportes por período, registrar las fechas exactas de la evidencia; los de emisión usados incluyen el 12-10-2026 y los de pagos el 13-10-2026.

Diego: integración y coordinación

Tu responsabilidad principal

• Revisar cómo la interfaz llama a la API y cómo la API ejecuta los procedimientos almacenados.

• Revisar parámetros, validaciones del servidor, manejo de errores y resultados mostrados en pantalla.

• Comentar las funciones importantes: propósito, entradas, salida, reglas y decisiones. Mantener la explicación coherente con el código final.

• Integrar las correcciones en pruebas-diego y comprobar los recorridos afectados.

Cómo evitar que dos personas se pisen

Zona	Responsable principal	Coordinación
SQL y modelo	Compañero 1	Diego revisa cambios funcionales y compatibilidad con la API.
Matriz, capturas y manual	Compañero 2	Recibe explicaciones técnicas de los otros dos.
API e interfaz	Diego	Recibe errores reproducibles y propuestas.
Seguridad	Diego + compañero 1	El compañero 2 documenta los casos y resultados acordados.
Exposición	Los tres	Cada uno enseña su área y aprende el recorrido completo.

Secuencia de revisión

Cada compañero entrega primero un pequeño ejemplo: un procedimiento documentado o una prueba completa. Diego revisa el formato antes de que continúen con todo el módulo. Después, cada entrega debe reunir cambios relacionados y evidencia de su comprobación.

Acordar quién edita los documentos compartidos. Si otro necesita aportar contenido, lo entrega al responsable o coordina el cambio. Un comentario de código no debe ocultar un cambio en el comportamiento.

Para aceptar una entrega

Revisar archivos y diferencias; comprobar que no contiene datos privados; ejecutar las comprobaciones relacionadas con el cambio; resolver observaciones; integrar y volver a comprobar el recorrido afectado. Una compilación correcta no sustituye probar una regla del negocio.

Este reparto no fija fechas ni nombres reales de los compañeros. Completen esos datos antes de empezar y acuerden un momento para la revisión conjunta y otro para ensayar la exposición.

Git: preparar la copia y la rama

Los comandos se ejecutan uno por uno en una terminal, dentro de la carpeta del repositorio. No copiar los textos de ejemplo sin sustituirlos. Las ramas siguientes son propuestas, no ramas creadas por este PDF.

1. Acceso e identidad

Tener Git instalado y una cuenta de GitHub con permiso de escritura en JOSUETUMG/MAGISAUTO. Si no tienen permiso, pedir acceso al administrador. Cada persona usa su propia cuenta e identidad; no la de Diego.

```
git --version
git config --global user.name "Nombre Apellido"
git config --global user.email "CORREO_DE_TU_CUENTA"
```

El correo puede ser el correo privado noreply proporcionado por GitHub. --global configura la identidad del usuario de esa computadora; en una máquina compartida, configurar solo el repositorio, sin --global, después de entrar en él.

2. Descargar el repositorio por primera vez

Ejecutar desde una carpeta donde quieran guardar el proyecto. Si ya tienen una copia, no volver a clonarla encima: entrar en la copia existente y comprobar su estado.

```
git clone https://github.com/JOSUETUMG/MAGISAUTO.git
cd MAGISAUTO
git remote -v
git status
git fetch origin
```

Confirmar que origin apunta al repositorio correcto y que no hay trabajo local pendiente. Diego debe haber confirmado el commit común que incluye la versión a revisar.

3. Crear la rama propia desde la de Diego

Compañero 1: ejecutar únicamente su bloque.

```
git switch -c companero1-sql origin/pruebas-diego
git branch --show-current
```

Compañero 2: ejecutar únicamente su bloque.

```
git switch -c companero2-pruebas-docs origin/pruebas-diego
git branch --show-current
```

El último comando debe mostrar la rama propia. -c crea una rama: se usa una sola vez. Si ya existe, usar git switch con su nombre, sin -c. No usar -C para sobreescribir una rama. La copia local contiene archivos; los datos y credenciales de SQL requieren configuración aparte según el README.

Git: guardar y subir el trabajo

4. Revisar antes de guardar

```
git branch --show-current
git status
git diff
```

Confirmar la rama propia y revisar cada archivo modificado. No subir contraseñas, cadenas privadas de conexión, appsettings.Local.json, credenciales exportadas, bin, obj ni datos personales reales en capturas. No incluir cambios ajenos por accidente.

5. Preparar archivos y crear un commit

Agregar solo los archivos de la entrega. La ruta siguiente es un ejemplo: reemplazarla por la ruta real, relativa al repositorio. Repetir git add por cada archivo necesario.

```
git add -- "RUTA/DEL/ARCHIVO"
git diff --cached
git diff --cached --check
git status
git commit -m "Documentar reglas de pagos y anulaciones"
```

Para pruebas, un mensaje útil sería "Documentar pruebas de reportes y evidencias". Si prepararon por error un archivo, git restore --staged -- "RUTA/DEL/ARCHIVO" lo quita del próximo commit y conserva su contenido local. Revisar también los archivos nuevos: git diff normal no muestra los que aún no están registrados.

6. Publicar la rama propia

Compañero 1:

```
git push -u origin companero1-sql
```

Compañero 2:

```
git push -u origin companero2-pruebas-docs
```

Después del primer envío, pueden usar git push desde esa misma rama. **Commit guarda localmente; push publica en GitHub.** No afirmar que se subió si aparece un error de autenticación o permisos.

7. Confirmar que llegó

```
git log -1 --oneline
git status -sb
```

Abrir GitHub, seleccionar la rama propia y comprobar el último commit y sus archivos. Enviar a Diego el enlace de la rama o del commit. Un mensaje local de trabajo limpio no demuestra por sí solo que se publicó. Referencia: documentación oficial de git push, enlazada al final.

Git: actualizar y pedir revisión

8. Recibir los cambios recientes de Diego

Primero guardar el trabajo propio en un commit y comprobar un estado limpio. Cambiar a la rama propia si no están en ella. Ejecutar:

```
git fetch origin
git merge origin/pruebas-diego
```

Esto incorpora los cambios de Diego en la rama actual. Volver a probar y luego ejecutar git push. Si Git dice que ya está actualizado, no hace falta crear otro commit. No usar push --force para evitar resolver una diferencia.

9. Si aparece un conflicto

1. Leer git status y localizar los archivos en conflicto.
2. Abrirlos, comprender ambas versiones y acordar la solución con el responsable.
3. Resolver los bloques, quitar las marcas de conflicto y conservar el resultado correcto.
4. Agregar cada archivo resuelto con git add, revisar, probar y completar con git commit.
5. Publicar con git push.

No aceptar automáticamente "todo lo mío" o "todo lo de ellos". Si no pueden resolverlo, coordinar con Diego; mientras la fusión siga en curso, git merge --abort permite cancelarla. Esta es la razón para empezar con el trabajo guardado y el estado limpio. No usar reset --hard ni clean -fd como solución rutinaria.

10. Abrir una solicitud de revisión en GitHub

En el repositorio, abrir Pull requests y crear una nueva comparación. Verificar expresamente: **base = pruebas-diego** y **compare = la rama del compañero**. Revisar Files changed antes de crearla; no aceptar main por defecto.

La solicitud debe explicar qué problema resuelve, qué cambia, cómo se probó y qué queda pendiente. Pedir revisión a Diego. Mientras no esté integrada, nuevos commits enviados a esa misma rama actualizan la solicitud.

11. Integración y problemas de acceso

Diego revisa, solicita ajustes si hacen falta e integra en pruebas-diego. Después actualiza su copia con estado limpio: git switch pruebas-diego y git pull --ff-only origin pruebas-diego. Si hay divergencia, se revisa antes de continuar. Nadie integra en main en este flujo.

Si push pide autenticación, completar el inicio de sesión con la cuenta autorizada mediante el administrador de credenciales o la aplicación Git utilizada. Ante un 403, verificar cuenta y acceso; no compartir contraseñas o tokens. Si el rechazo dice non-fast-forward, traer y revisar los cambios de la rama remota propia antes de fusionar y volver a enviar.

Plantillas para entregar el trabajo

Completar estas plantillas con información real. Los nombres de archivos sugeridos son propuestas; acuerden dónde guardarlos antes de crear documentos duplicados.

Informe breve del compañero 1

Responsable / rama / commit: ...
Procedimientos revisados y ubicación: ...
Reglas y relaciones explicadas: ...
Comentarios añadidos: ...
Cálculos comprobados y evidencia: ...
Hallazgos y correcciones propuestas: ...
Pruebas realizadas / pendientes: ...

Informe breve del compañero 2

Responsable / rama / commit: ...
Módulos y casos ejecutados: ...
Evidencias anteriores revisadas: ...
Casos correctos, fallidos y pendientes: ...
Errores reproducibles: ...
Índice de capturas y guía: ...
Límites de la revisión: ...

Descripción de una solicitud de revisión

Problema y resultado: qué se necesitaba y cómo queda.
Cambios: archivos, funciones o procedimientos relevantes.
Validación: entorno, casos ejecutados, resultados y evidencias.
Pendientes: lo que aún requiere revisión o no se probó.
Destino: pruebas-diego.

Archivos propuestos para organizarse

Responsable	Contenido sugerido
Compañero 1	Un documento de revisión SQL y el diagrama; cambios comentados en las fuentes correspondientes.
Compañero 2	Una matriz de pruebas, guía de usuario e índice de evidencias; capturas agrupadas por caso.
Diego	Registro de integración y observaciones resueltas; documentación técnica de API e interfaz.

Control antes de enviar

Confirmar rama y destino; revisar todos los archivos; no incluir secretos; indicar qué se probó; explicar el cambio con palabras propias; comprobar el push en GitHub; enviar el enlace de revisión. No basta entregar capturas sueltas o decir "ya quedó".

Preparación para la exposición

No memorizar respuestas sin relacionarlas con el código. Cada integrante debe mostrar una parte del sistema y luego responder preguntas de las otras áreas.

Pregunta	Qué debe contener la explicación
¿Cómo viaja una operación?	Pantalla, petición a la API, validación, procedimiento SQL y respuesta.
¿Por qué se utilizan procedimientos?	Requisito del proyecto y lógica implementada; mostrar un ejemplo real.
¿Por qué Q1,500 no reduce capital en Q1,500?	El pago se reparte entre capital, interés, manejo y seguros.
¿Qué ocurre al anular un pago?	Se conserva el registro y se revierten sus efectos según la fecha de corte.
¿Saldo de capital y saldo total son iguales?	No: Q78,708.91 y Q86,239.56 corresponden a conceptos distintos.
¿Cuándo empieza la mora?	En la prueba, al día siguiente del vencimiento; mostrar ambos cortes.
¿Qué significa Sin dato?	Ausencia de valor aplicable o registrado; no se debe convertir siempre en cero.
¿La comisión se pagó al vendedor?	El reporte acredita generación/reversión; no demuestra un desembolso.
¿La depreciación reduce la deuda?	No; compara una valoración del vehículo con el capital pendiente.
¿Cómo comprobaron la seguridad?	Mostrar controles y pruebas efectivamente revisados y reconocer pendientes.
¿Qué diferencia hay entre commit y push?	Guardar una versión local frente a publicarla en el repositorio remoto.

Ensayo conjunto

Compañero 1 explica aprobación y reparto de un pago. Compañero 2 demuestra una validación y localiza la evidencia. Diego sigue la petición desde la interfaz hasta la base de datos. Después intercambian preguntas y cada uno explica una parte que no fue su responsabilidad principal.

Condición final de entrega

Los tres pueden explicar el flujo completo; los cambios están revisados e integrados en pruebas-diego; la publicación se verificó; las pruebas tienen evidencia; los errores pendientes están identificados. Los comentarios deben ayudar a comprender el proyecto, no atribuir comprobaciones que nadie realizó.

Referencias y aclaraciones

Guía de coordinación del equipo. No sustituye el README, los requisitos del curso ni la revisión del código que finalmente se entregue.

Repositorio y versión de trabajo

Repositorio verificado para esta guía:
https://github.com/JOSUETUMG/MAGISAUTO
Rama de integración: **pruebas-diego**. Los nombres companero1-sql y companero2-pruebas-docs son propuestas para acordar con el equipo.

Referencias oficiales para Git y GitHub

Crear y cambiar de rama:
https://git-scm.com/docs/git-switch

Publicar cambios en el remoto:
https://git-scm.com/docs/git-push

Fusionar ramas y resolver o cancelar una fusión:
https://git-scm.com/docs/git-merge

Crear una solicitud de revisión:
GitHub Docs: Creating a pull request

Alcance de las comprobaciones citadas

Los importes y ejemplos del proyecto provienen de las capturas y resultados compartidos en esta conversación. El documento organiza el trabajo pendiente y reproduce esos valores de referencia; su creación no ejecuta nuevas pruebas de la base de datos ni certifica que todos los escenarios posibles estén cubiertos.

El recorrido manual de las 20 consultas es un avance concreto. Siguen siendo tareas separadas documentar sus evidencias, revisar la seguridad, corregir detalles identificados, integrar cambios y preparar la exposición. La fase 6 no se declara cerrada por este PDF.

Datos a completar por el equipo

Nombre del compañero 1: _____
Nombre del compañero 2: _____
Commit común de partida: _____
Fecha acordada para revisión: _____
Fecha del ensayo conjunto: _____